Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(бакалаврская работа)
по направлению подготовки
09.03.01 — «Информатика и вычислительная техника»

GRADUATION THESIS
(Bachelor's Graduation Thesis)
Field of Study
09.03.01 — «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Visual Studio Code Extension for Type-Based Visualization for the Stella Programming Language / Расширение Visual Studio Code для визуализации информации на основе типов для языка программирования Stella**

Работу выполнил / Thesis is executed by	**Фадеенков Антон Олегович / Fadeenkov Anton Olegovich**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Кудасов Николай Дмитриевич / Kudasov Nikolay Dmitrievich**	подпись / signature

Иннополис, Innopolis, 2025

Contents

List of Tables

List of Figures

Abstract

This thesis presents a Visual Studio Code extension for visual analysis of programs written in Stella, an educational statically typed programming language. The problem addressed in this work is that standard editor diagnostics expose the final result of language analysis but do not show the intermediate structures used during parsing and type checking. The developed extension makes five kinds of information available inside the editor: an abstract syntax tree, a scope and typing-context view, a type derivation tree, a reverse type-error trace, and a type dependency graph. The implementation uses TypeScript, the Visual Studio Code Extension API, Langium, and custom Language Server Protocol requests. The client and server are separated so that semantic analysis remains on the language-server side, while editor-specific rendering is implemented on the client side. The prototype was evaluated by the author using six reproducible scenarios for correct and incorrect Stella programs. The evaluation confirmed the availability of the required data, synchronization with source ranges, bidirectional navigation, decomposition of typing steps, and highlighting of type conflicts. These results show that the prototype satisfies the functional criteria used in the evaluation. They do not prove that the extension improves learning outcomes, code readability, or debugging speed for other users because no controlled user study was conducted. A user study with objective task-completion measurements is therefore left for future work.

Chapter 1

Introduction

1.1 Motivation

Static type systems detect inconsistent operations before program execution and provide information used by compilers and development tools [1], [2], [3], [4]. For a student, however, the result of type checking is often visible only as a final type or an error message. The intermediate structures—the abstract syntax tree, the typing context, the applied typing rules, and dependencies between expressions—remain hidden.

This problem is relevant in courses on compilers and programming languages [1], [5]. Students are expected to connect source code with formal notation, for example a typing judgment of the form $\Gamma \vdash e : T$. In this notation, Γ is the typing context: a finite mapping from identifiers that are available at the current program point to their types. The expression e is assigned type T under this context. When only the final result is shown, it can be difficult to reconstruct which rule was applied and which subexpression caused a mismatch.

Visualization is one way to expose these internal structures. Studies of software, algorithm, and data-structure visualization report that interactive representations can support the formation of mental models and the analysis of abstract processes [6], [7], [8], [9], [10]. Similar motivation is used in tools such as Jeliot

and visAST, which connect program text with executable or structural representations for software-language education [11], [12]. These studies motivate the interface design in this thesis, but their results cannot be transferred automatically to a new tool or to type-system visualization. The educational effect of the developed extension therefore requires a separate evaluation.

1.2 Problem Statement

Stella is an educational language designed for studying the implementation of static type systems [13]. Its existing language server performs parsing, name resolution, type checking, and diagnostics. Standard Visual Studio Code features can display syntax highlighting, hover information, and error messages, but they do not provide a built-in representation of Stella typing derivations or semantic dependencies.

The practical problem addressed in this work is the absence of an integrated interface that connects Stella source code with the internal results of semantic analysis. A useful interface should answer the following questions:

- Which abstract syntax tree (AST) node corresponds to the current source fragment?
- Which variables, parameters, and functions are available at the cursor position?
- Which typing rules produce the type of the selected expression?
- Which sequence of checks leads to a reported type error?
- Which expressions and declarations influence the type of the selected expression?

A *type dependency graph* is used in this thesis to answer the last question. It is a directed graph whose nodes represent selected expressions, functions, parameters, local bindings, and related subexpressions. An edge $u \to v$ means that the type or type-checking result of v depends on information obtained from u. For example, in a function application, the argument and the called function have incoming edges to the application node. An edge is marked as conflicting when the connected types violate the relevant typing requirement.

1.3 Goal and Research Questions

The goal of this work is to design and implement a Visual Studio Code extension that exposes Stella syntax and type-analysis information as interactive visual representations linked to source code.

The work addresses two research questions:

RQ1: How can the semantic information produced by the Stella language server be represented in Visual Studio Code without mixing different analysis tasks into one overloaded interface?

RQ2: Does the implemented prototype satisfy the functional criteria of locality, hierarchy, source-code traceability, navigation, and explicit conflict indication in a set of reproducible Stella scenarios?

The research questions deliberately concern the design and functional behavior of the prototype. Claims about improved learning outcomes, perceived usability, code readability, or debugging speed would require a controlled study with representative users and are outside the evidence collected in this work.

1.4 Contributions

The contributions of this work are the following implemented artifacts and evaluation materials:

1. A synchronized AST viewer that connects tree nodes with source-code ranges.
2. A scope and context panel that groups available bindings by the program construct that introduces them.
3. An interactive type derivation view that represents premises, typing rules, conclusions, and the local context Γ.
4. A reverse type-error trace that narrows a diagnostic from an enclosing construct to the conflicting subexpression.
5. A type dependency graph that shows type-relevant relations between functions, arguments, branches, local bindings, and selected expressions.
6. A set of custom Language Server Protocol requests and shared data structures used to transfer analysis results from the language server to the extension client.
7. A set of reproducible validation scenarios and example Stella programs for the implemented views.

1.5 Thesis Structure

Chapter 2 reviews educational visualization, Stella, and the technologies used by language tools. Chapter 3 defines the design requirements and evaluation method. Chapter 4 describes the architecture, technology choices, implementation, and evolution of the prototype. Chapter 5 presents the results before discussing the

individual validation scenarios and threats to validity. Chapter 6 summarizes the answers to the research questions and identifies future work. Appendix A lists the implementation artifacts and reproduction steps.

Chapter 2

Background and Related Work

2.1 Visualization in Computer Science Education

Program visualization is used to make structures and execution processes observable and has been classified as a distinct area of software visualization research [6], [9]. Kogan, Chassidim, and Rabaev evaluated AVDS (Animation and Visualization of Data Structures) in a data-structures course [10]. Their experiment involved 78 students. More than 75% of the students who used AVDS received grades above 80 points, and the reported mean System Usability Scale score was 79.17.

The System Usability Scale (SUS) is a ten-item questionnaire used to obtain a single perceived-usability score in the range from 0 to 100 [14], [15], [16]. The result is an index, not a percentage of correctly completed tasks. Therefore, the value 79.17 describes participants' subjective assessment of AVDS usability; it does not directly measure learning or prove the usability of another visualization tool.

Aalvik's visAST project applies visualization to abstract syntax trees [12].

Its main purpose is to help students connect source text, grammar concepts, and the hierarchical AST representation. The project is particularly relevant to this thesis because it treats source-to-structure navigation as part of the learning interface. At the same time, visAST focuses on syntax, while this work also visualizes scope, typing derivations, errors, and dependencies.

The related studies support the general motivation for visualization and program comprehension support [17], [18], but they do not provide empirical evidence for the particular Stella extension developed in this work. For this reason, their findings are used as design background rather than as proof of the effectiveness of the prototype.

2.2 Stella and Static Type Systems

Stella (Statically Typed Extensible Language for Learning Advanced Compiler Construction) is designed for teaching the implementation of type systems [13], [19]. The language supports a core calculus and extensions that introduce additional type-system features. This structure allows course assignments to progress from basic typing rules to more advanced language constructs.

A type checker can be described using judgments such as

$$\Gamma \vdash e : T,$$

where Γ denotes the typing context, e is an expression, and T is its type [1], [2], [20]. The context records assumptions available at the current point, for example $x : \texttt{Nat}$. A derivation tree explains a judgment by showing the premises required by a typing rule and the conclusion produced by that rule.

An AST, a typing context, and a derivation tree answer different questions. The AST describes syntactic composition. The context describes available identifiers and their types. The derivation tree explains why an expression has a type.

A type dependency graph provides another view: it emphasizes relations between type-relevant program elements instead of reproducing the complete proof structure. Keeping these models separate is an important design decision of this work.

2.3 Language Server Protocol

The Language Server Protocol (LSP) standardizes communication between an editor client and a language server and uses JSON-RPC as the underlying message format [21], [22]. The protocol defines common operations such as diagnostics, completion, hover information, and navigation to definitions. This separation allows language analysis to remain independent from a particular editor.

The standard LSP methods do not define responses for Stella-specific derivation trees, scope frames, or type dependency graphs. However, LSP permits custom requests. The implementation in this work uses this extension point: the server produces semantic data, shared TypeScript interfaces define the response format, and the Visual Studio Code client renders the result. This preserves the client–server boundary while supporting domain-specific visualizations.

After this definition, the shortened form *LSP* is used consistently.

2.4 Visual Studio Code Extension API

The Visual Studio Code Extension API provides commands, editor events, tree views, decorations, and webviews [23], [24], [25]. Tree views are suitable for structures that naturally use parent–child navigation, such as an AST or nested scope frames. Webviews provide more control over layout and interaction and are therefore suitable for proof-like derivation trees, error traces, and dependency graphs.

The API also provides source positions and ranges. These ranges are essential for the design in this thesis because every meaningful visual node can be

connected to the corresponding fragment of Stella code. The same mechanism supports both directions of navigation: an editor selection updates a view, and a click in a view reveals the source fragment.

2.5 Research Gap

Existing work provides evidence that visualization can be useful in computer science education and demonstrates tools for algorithm, program, and AST visualization [7], [8], [10], [11], [12]. Stella provides a structured environment for learning type-system implementation [13]. LSP and the Visual Studio Code Extension API provide the technical basis for integrating language analysis with an editor [21], [23].

The remaining gap is a Stella-specific interface that exposes several stages of type analysis and keeps them connected to source code. The gap is practical rather than theoretical: this work does not introduce a new typing algorithm. It transforms existing syntax and type information into coordinated views for program inspection. The next chapter defines the requirements used to design and evaluate these views.

Chapter 3

Design and Methodology

3.1 Design Objective

The extension is designed as an explanatory layer over the existing Stella language services. It does not replace parsing or type checking. Instead, it requests analysis results from the language server and presents them in a form connected to the editor. This boundary is important because a visualization must not introduce a second, inconsistent implementation of the Stella type system.

The design starts from user questions rather than from available UI components. When reading code, a user may need to understand its syntactic structure, the names visible at a position, the derivation of a type, the source of a diagnostic, or the data that influences a type-checking result. These tasks use related information but require different representations.

3.2 Design Requirements

Five functional requirements were defined before evaluating the prototype.

The requirements are intentionally observable. Terms such as “easy to understand” or “readable” depend on a user’s experience and should be measured in a user study or interpreted through established usability and visualization prin-

Table 3.1. Design requirements for the visual analysis extension

ID	Requirement	Observable condition
R1	Locality	A view is built for the active source position or selected expression and avoids unrelated program data.
R2	Hierarchical explanation	A compound expression is decomposed into smaller AST nodes, typing premises, or diagnostic steps.
R3	Source traceability	Every meaningful visual element contains a source range when such a range exists.
R4	Bidirectional navigation	Editor selection updates a view, and selecting a visual element reveals the corresponding source fragment.
R5	Explicit conflict indication	Type mismatches are distinguished from valid dependencies and ordinary nodes.

ciples [26], [27]. In contrast, it is possible to verify whether a graph contains a conflict edge or whether a click reveals the expected source range.

3.3 Separation of Visual Representations

The prototype uses five coordinated views.

3.3.1 Abstract Syntax Tree

The AST view represents the hierarchical composition of the program. It is suitable for answering which language construct contains the cursor and how a compound expression is divided into child nodes. A native tree interface supports expansion and collapse and preserves a familiar editor interaction.

3.3.2 Scope and Typing Context

Scope and typing context are closely related but not identical concepts. Scope determines which declarations are visible at a program position. The typing context Γ associates the visible names with their types. In the interface, this information is represented as nested frames. A frame identifies the construct that introduced bindings, such as the program, a function, a lambda abstraction, a `let` expression, or a pattern.

For example, in

```
fn(x: Nat) => let y = succ(x) in y
```

the context at the final occurrence of `y` contains the function parameter $x : \texttt{Nat}$ and the local binding $y : \texttt{Nat}$. Showing the bindings in separate frames preserves their origin instead of flattening them into one list.

3.3.3 Type Derivation Tree

A type derivation tree represents a judgment as a conclusion supported by zero or more premises. For the expression `succ(x)`, where $x : \texttt{Nat}$ is available, a simplified derivation is

$$\frac{\Gamma \vdash x : \texttt{Nat}}{\Gamma \vdash \texttt{succ}(x) : \texttt{Nat}}\ \text{T-Succ.}$$

The view therefore needs a proof-oriented layout rather than a conventional vertical list. Each node stores the rule name, conclusion, premises, expression text, inferred type, relevant part of Γ, and source range.

Only bindings relevant to the current expression should be displayed when possible. Repeating the complete program context at every node creates long judgments and hides the rule being explained. This is a design choice for locality, not a change to the formal typing relation.

3.3.4 Reverse Type-Error Trace

The error-trace view starts from a diagnostic and follows a narrowing sequence toward the conflicting expression. For

```
if true then 0 else false
```

the trace should show the conditional-expression check, the requirement that both branches have compatible types, and the observed branch types `Nat` and `Bool`. Syntax diagnostics are excluded because parser-token expectations do not form a type derivation and would produce misleading output.

A chain is used instead of a complete derivation tree. The purpose is to retain the diagnostic path that is relevant to the current mismatch rather than show every valid part of the expression.

3.3.5 Type Dependency Graph

The type dependency graph is a directed graph $G = (V, E)$. The idea is related to program dependence graphs and slicing, but the graph in this work is local and type-oriented rather than a complete program-dependence representation [28], [29], [30]. The set V contains type-relevant program entities: functions, parameters, local bindings, cases, expressions, and the selected node. An edge $(u, v) \in E$ states that checking the type of v uses information associated with u. Edges may have labels such as `argument`, `condition`, `then`, `else`, `value`, or `return`.

For the application `takesNat(flag)`, the function declaration and the argument are dependencies of the application. If `takesNat` expects `Nat` but `flag` has type `Bool`, the argument edge and the related node are marked as conflicts. The graph does not claim to be a complete program-dependence graph. It is a local explanatory graph constructed for the selected expression and its type-checking relations.

3.4 Connection to Source Code

A shared range representation is used across all custom responses. A range contains start and end positions in a source document. The client converts this representation to a Visual Studio Code range and uses it for highlighting and navigation.

This design establishes a common interaction rule. Moving the cursor selects the deepest relevant AST node and refreshes position-dependent views. In the opposite direction, clicking an AST item, scope binding, derivation step, trace step, or dependency-graph node reveals its range in the editor. The views therefore remain secondary representations of the same source document rather than independent diagrams.

3.5 Client–Server Boundary

The language server owns parsing, AST traversal, scope computation, type inference, and construction of semantic relations. The extension client owns command registration, editor event handling, view state, layout, and source highlighting. Shared TypeScript interfaces define the protocol boundary.

This separation follows the general LSP architecture [21]. It also reduces the risk of semantic disagreement: the client renders types returned by the server instead of implementing type rules again. Custom requests are used because the standard protocol has no messages for the five Stella-specific representations.

3.6 Evaluation Methodology

The prototype is evaluated through scenario-based functional validation, following the general principle that software-engineering evaluations should make their procedure, context, and validity limitations explicit [31], [32]. Each scenario

defines an input program, a cursor position or selected expression, an action, and an observable expected result. The scenarios cover both correct programs and programs with type mismatches.

The method answers RQ2 by checking the requirements in Table 3.1. It can confirm, for example, that selecting a derivation node reveals a source range and that incompatible conditional branches are marked in a graph. It cannot determine whether a representative student completes a task faster, makes fewer mistakes, or reports a better usability score.

The author executed the scenarios manually in the extension development environment and compared the observed state with the expected result. Example programs and expected graph behavior are included with the artifact and listed in Appendix A. The evaluation results and limitations are reported in Chapter 5.

Chapter 4

Implementation

4.1 Architecture

The implementation consists of three layers. The first layer is the Stella language server. It parses a document, resolves names, performs type analysis, and handles custom requests. The second layer contains shared protocol and analysis types. These interfaces describe AST nodes, scope frames, derivation nodes, trace steps, dependency nodes, edges, and source ranges. The third layer is the Visual Studio Code extension client, which requests data and renders tree views or webviews.

The extension registers commands for showing the derivation tree, error trace, dependency graph, and AST in JSON form. It also creates persistent AST and scope views in the Stella activity-bar container. Editor-selection events are used to keep position-dependent information synchronized with the active document.

4.2 Technology Stack and Rationale

Table 4.1 summarizes the selected technologies and their relation to the design requirements.

Table 4.1. Technology stack and design rationale

Technology	Role	Reason for selection
TypeScript	Client, server, and shared protocol types	One type system across the boundary makes request and response structures explicit and reduces accidental disagreement between layers [33].
Langium	Stella grammar, AST, documents, and language services	The existing Stella server already uses Langium, so the visual analysis can reuse parsed documents and language-service infrastructure instead of introducing another parser [34].
LSP client library	Communication between the editor and server	The protocol preserves separation between semantic computation and presentation and supports custom Stella requests [21], [22].
Visual Studio Code Extension API	Commands, events, tree views, decorations, and panels	The target users work in Visual Studio Code, and the API provides direct access to source ranges and native editor interaction [23], [24], [25].
HTML, CSS, and JavaScript in webviews	Derivation, trace, and graph layout	These views need proof rules, cards, directed edges, and click handling that cannot be expressed clearly with a standard tree view.

The Stella services also use Typir as type-system infrastructure on top of Langium [35]. The stack is therefore aligned with the design. TypeScript and shared interfaces support a stable client–server contract. LSP enforces architectural separation. Native tree views are used for hierarchical navigation, while webviews are limited to representations that require custom layout. Source ranges are retained in every layer to satisfy traceability and navigation requirements.

4.3 Custom LSP Requests

The implementation introduces the following custom methods:

- `stella/syntaxTree` returns the document AST in a serializable form;
- `stella/scope` returns the active node and nested scope frames;
- `stella/typeDerivation` returns a recursive proof-like derivation;
- `stella/typeErrorTrace` returns a diagnostic and an ordered trace;
- `stella/typeDependencyGraph` returns graph nodes, edges, labels, types, and conflict markers.

Each request contains a document URI and, where required, a cursor position. The server resolves the current Langium document and selects the smallest relevant AST node. A null response is allowed when the cursor does not correspond to a visualizable construct. This behavior lets the client show a clear placeholder instead of failing.

4.4 AST Visualization

The AST is rendered with the Visual Studio Code tree-view API. Each serialized node contains an identifier, a label, child nodes, and a source range. The

synchronization controller searches for the deepest node that contains the cursor position. Selecting an item performs the reverse operation and reveals the stored range.

Figure 4.1 shows the AST panel next to the corresponding Stella source. The highlighted `Abstraction` node and highlighted source region demonstrate the source-to-tree connection. The hover text also displays the selected AST node type and a compact subtree.

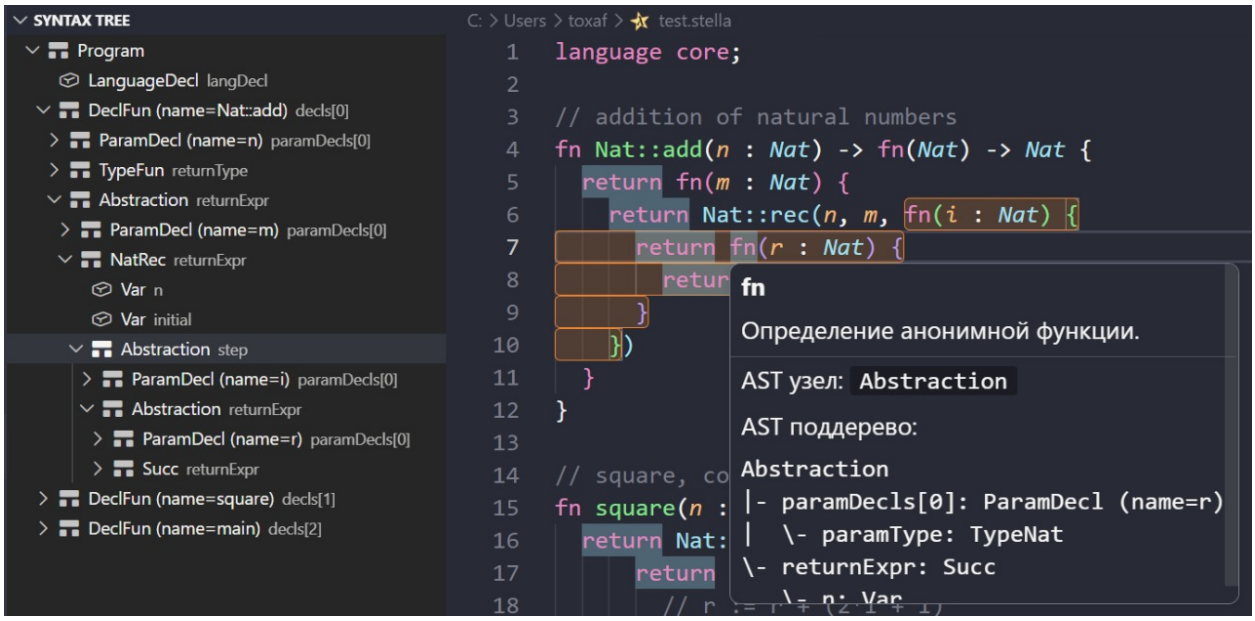

Figure 4.1. Synchronized AST view for a Stella function. The selected `Abstraction` node in the tree corresponds to the highlighted nested function in the editor; the hover panel shows the node kind and a textual subtree.

4.5 Scope and Context View

The server constructs scope frames from ancestors of the active AST node. Separate handlers collect program declarations, function parameters, local declarations, `let` bindings, recursive bindings, and variables introduced by patterns. Type labels are obtained from explicit annotations or inferred type information when available.

The client renders each frame as a collapsible group and each binding as a child item. Figure 4.2 shows three frames: a lambda frame, a function frame, and

the program frame. This presentation makes it possible to distinguish the local parameter `m`, the outer function parameter `n`, and globally declared functions.

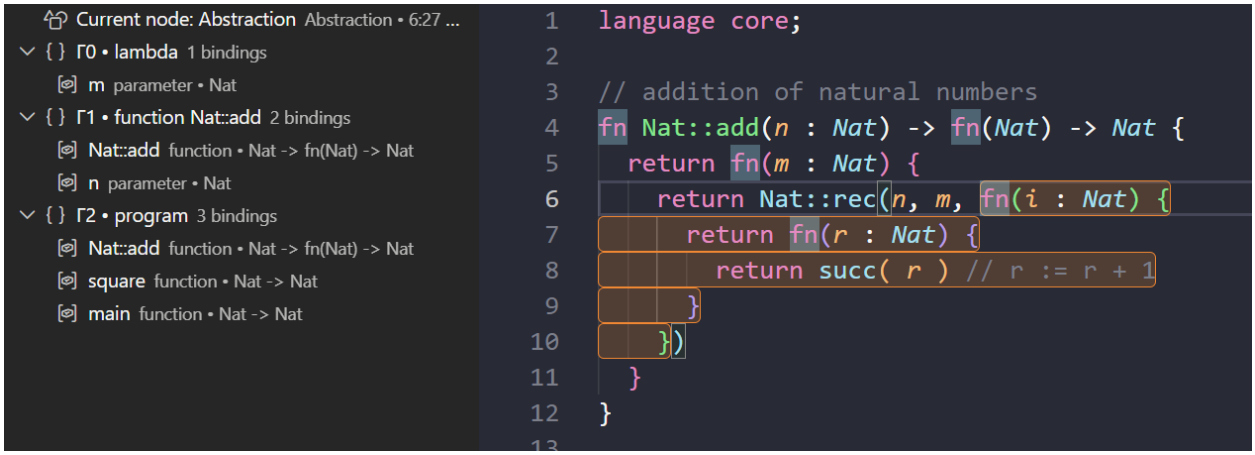

Figure 4.2. Scope and typing-context view at a nested Stella abstraction. Frames Γ_0, Γ_1, and Γ_2 separate bindings introduced by the lambda, the enclosing function, and the program; each binding includes its role and type.

4.6 Type Derivation View

The server builds a recursive `TypeDerivationNode`. A node contains a rule name, a conclusion, zero or more premises, a source range, expression text, inferred type, and context text. The client renders premises above a horizontal rule and the conclusion below it.

Figure 4.3 shows a derivation for nested abstractions ending in `succ(r)`. The path includes T-Var, T-Succ, and two T-Abs applications. A click on any rule sends its range back to the extension and highlights the corresponding source fragment.

4.7 Type-Error Trace

The error-trace command first chooses a type-related diagnostic near the cursor. Parser diagnostics and token-expectation messages are filtered out. The server

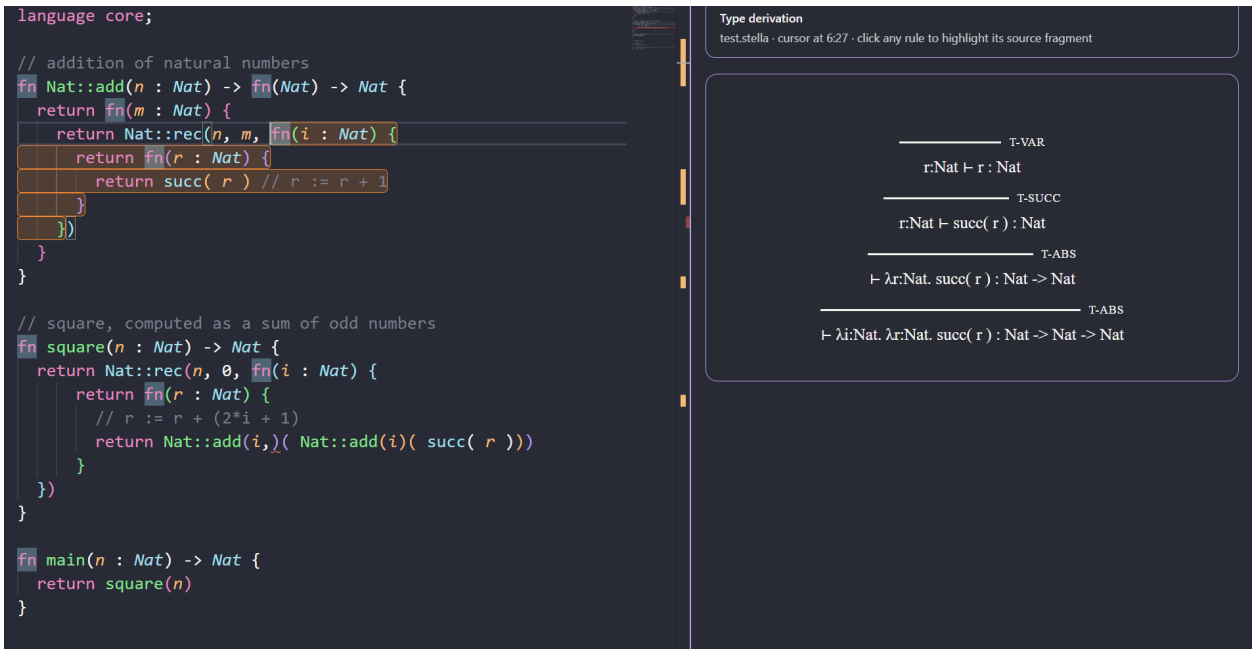

Figure 4.3. Interactive type derivation for nested Stella abstractions. Premises are placed above rule lines, conclusions below them, and the editor highlights the source fragment associated with the selected derivation step.

then builds a sequence of related expressions and checks. Each step contains a rule name, judgment, explanation, type information, and source range.

Figure 4.4 shows a function declared to return `Bool` whose body `succ(n)` has type `Nat`. The first card states the function contract, and the next card identifies the computed body type. The red border distinguishes the conflict path from an ordinary derivation.

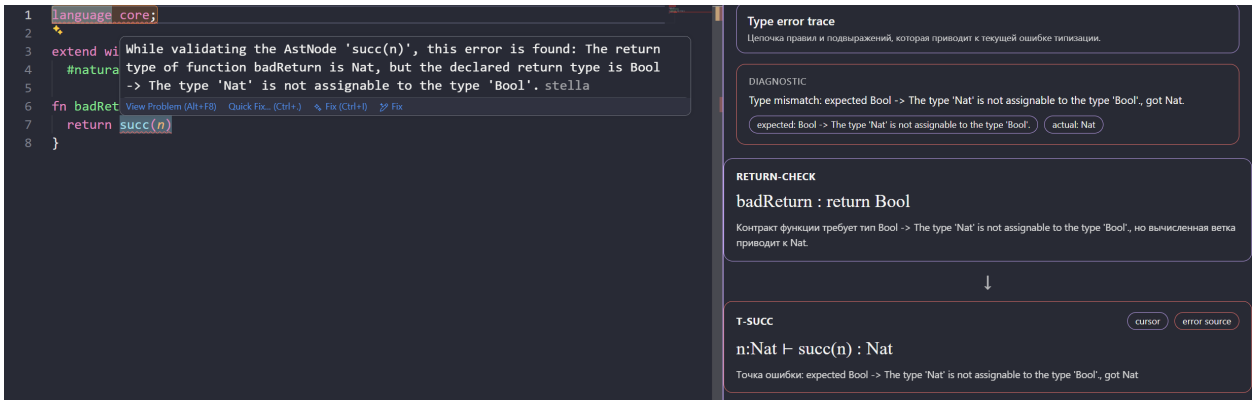

Figure 4.4. Reverse trace for a return-type mismatch. The trace connects the declared `Bool` result of `badReturn` with the T-SUCC judgment that gives `succ(n)` type `Nat`; the conflicting source expression is highlighted.

4.8 Type Dependency Graph

The dependency-graph server handler creates nodes for the selected expression and relevant functions, parameters, bindings, cases, and child expressions. Nodes are assigned layers so that dependencies are displayed from left to right. Directed edges are labeled according to their role in type checking.

For a function application, the graph connects the called function and each argument to the application. For a conditional expression, it creates edges for the condition, the `then` branch, and the `else` branch. For a `let` expression, the value flows into its binding, and references to that binding flow into later expressions. The enclosing function is connected through a `return` relation.

Figure 4.5 illustrates the graph structure on a simplified argument-mismatch example. The actual webview uses the same logical model, but renders the nodes as clickable elements and applies visual conflict styling to the marked relation.

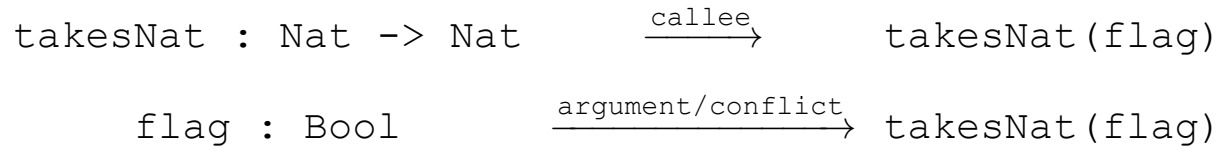

Figure 4.5. Simplified type dependency graph for an argument mismatch. The application depends on the called function and on the argument expression; the argument edge is marked as conflicting because `flag` has type `Bool`, while `takesNat` expects `Nat`.

The server compares available type labels with rule expectations. A mismatching argument or incompatible pair of branches sets `isConflict` on the corresponding nodes and edges. The webview uses the Visual Studio Code error color for these elements. Clicking a graph node reveals its source range, so the graph remains connected to the selected program.

4.9 Evolution of the Prototype

The implementation was developed in several iterations.

The first iteration exposed the AST and synchronized it with the cursor. This established stable node identifiers and source ranges. The second iteration added scope information. An initial flat list of names was replaced with nested frames after it became clear that a flat list did not explain where a binding originated.

The third iteration introduced the type derivation tree. The first version repeated a large context in each judgment and sometimes stopped decomposition at expressions that were still too complex. The displayed context was then reduced to bindings relevant to a subexpression, and additional expression cases were decomposed into premises. This change reduced repeated context and increased the number of displayed derivation steps, although its effect on human task performance was not measured.

The fourth iteration added reverse error tracing. Early output included parser diagnostics, which produced long token-expectation messages unrelated to typing. Diagnostic filtering and shorter explanations were added to keep the trace focused on type conflicts.

The final iteration added the type dependency graph. This view complements the derivation tree: the derivation emphasizes formal rules, while the graph emphasizes relations and conflict propagation. The iterations show that the implementation evolved from structural navigation toward semantic explanation, while reusing the same source-range and custom-request infrastructure.

4.10 Chapter Summary

The implementation provides five separate but coordinated visualizations. Semantic data is computed on the server, transferred through typed custom LSP requests, and rendered by the Visual Studio Code client. The next chapter eval-

uates whether these components satisfy the observable requirements defined in Chapter 3.

Chapter 5

Evaluation

5.1 Results Summary

The prototype was checked by the author using six manual scenarios. Table 5.1 presents the results first; the procedure and interpretation follow in the next sections.

Table 5.1. Results of the scenario-based functional evaluation

ID	Scenario	Observed result	Status
S1	AST synchronization	Cursor and tree selection revealed matching source ranges.	Pass
S2	Context inspection	Bindings were grouped by introducing scope and displayed with types.	Pass
S3	Type derivation	Named rules, premises, conclusions, and source links were displayed.	Pass
S4	Error trace	The mismatch path was shown, while parser diagnostics were excluded.	Pass
S5	Dependency graph	Labeled dependencies and type conflicts were displayed.	Pass
S6	View separation	Five task-specific representations remained separate.	Pass

The six scenarios satisfied the functional criteria R1–R5 for the tested examples. This confirms that the prototype exposes information needed for structural and type-oriented inspection and connects this information to source code.

Following standard practice in empirical software engineering, the result must be interpreted narrowly [31], [32]. It does not demonstrate that users read code faster, find errors faster, or learn type systems better. No participant group, control condition, completion-time measurement, accuracy measurement, or SUS questionnaire was used [14], [15]. The evaluation therefore validates the implementation behavior, not a general improvement in readability or debugging effectiveness.

5.2 Evaluation Setup

The evaluation used the development version of the Visual Studio Code extension and small Stella programs with known expected structures. The author opened each program, placed the cursor on the specified expression, invoked the corresponding command, and compared the resulting view with the expected nodes, relations, type labels, and source ranges.

Correct programs were used for AST, context, and derivation scenarios. Incorrect programs were used for return-type, function-argument, and conditional-branch mismatches. The input files, expected graph behavior, and error-trace examples are listed in Appendix A.

5.3 Scenario Details

5.3.1 S1: AST and Editor Synchronization

The AST scenario used a Stella program with nested functions and recursive natural-number operations. The cursor was moved between an outer function, a nested abstraction, and a variable reference. For every position, the AST panel selected the deepest node containing the cursor. Selecting the node in the tree revealed and highlighted the corresponding source range.

This scenario checks locality (R1), source traceability (R3), and bidirectional navigation (R4). Figure 4.1 records one observed state.

5.3.2 S2: Scope and Context Inspection

The context scenario used nested parameters and local bindings. The expected result was not only a list of visible names but also separation by origin. At a nested abstraction, the panel displayed the local lambda parameter, parameters of the enclosing function, and program declarations in different frames. This output

was treated as a concrete editor representation of the typing context Γ: each frame contributed assumptions of the form $x : T$ that were available for the selected expression.

The scenario checks locality (R1), hierarchical explanation (R2), and source traceability (R3). Figure 4.2 shows the frames and type labels observed in the panel.

5.3.3 S3: Type Derivation Inspection

The derivation scenario selected an expression ending in `succ(r)` inside nested abstractions. The expected path contained a variable rule, the successor rule, and abstraction rules. The webview displayed these premises and conclusions in proof form. Clicking a rule highlighted the source fragment associated with that derivation node.

This scenario checks hierarchical explanation (R2), source traceability (R3), and bidirectional navigation (R4). The observed derivation is shown in Figure 4.3.

5.3.4 S4: Reverse Type-Error Trace

Two mismatch forms were checked. In the first, a function declared a `Bool` return type but returned `succ(n)` of type `Nat`. In the second, an `if` expression returned `Nat` in one branch and `Bool` in the other. The trace started from the relevant contract or conditional check and narrowed to the conflicting expression.

A syntax-error example with an unclosed call was also opened. The expected behavior was a placeholder rather than a long type trace generated from parser token expectations. This scenario checks locality (R1), hierarchy (R2), source traceability (R3), and conflict indication (R5). Figure 4.4 shows the return-type case.

5.3.5 S5: Type Dependency Graph

Three programs were used:

1. In `takesNat(flag)`, the function expects `Nat` while `flag` has type `Bool`. The argument relation was marked as conflicting.
2. In `if flag then 0 else false`, condition, `then`, and `else` edges were displayed. The condition was valid, while the incompatible branches were marked as a conflict.
3. In nested `let` expressions, value expressions were connected to binding nodes, and the bindings were connected to the final arithmetic expression.

Each clickable graph node revealed its source fragment. This scenario checks locality (R1), source traceability (R3), bidirectional navigation (R4), and explicit conflict indication (R5).

5.3.6 S6: Separation of Views

For a correct program, the AST, context, and derivation views were opened. For an incorrect program, the error trace and dependency graph were added. Each view retained one primary role: syntax hierarchy, visible bindings, formal derivation, diagnostic narrowing, or dependency relations.

This scenario supports the design decision not to combine all semantic information into a single panel. It checks hierarchical explanation (R2) and keeps each task small enough to inspect independently.

5.4 Comparison with the Standard Editor Output

Table 5.2 compares information available through standard editor feedback with information exposed by the prototype. The comparison concerns interface capability, not user performance.

Table 5.2. Functional comparison of standard feedback and the developed views

Task	Standard feedback	Developed extension
Inspect syntax	Source text and syntax highlighting	Expandable AST linked to exact source ranges
Inspect available names	Completion and definition navigation	Frames showing origin, role, and type of active bindings
Explain a type	Final type in hover or diagnostic	Premises, rule names, context, and conclusion
Analyze a mismatch	Diagnostic message and underlined range	Reverse trace and conflict-marked dependencies
Follow type relations	No Stella-specific graph	Local directed graph with labeled dependency edges

The extension makes intermediate analysis data directly observable and reduces the need to reconstruct that data manually from a final message. Whether this capability produces a measurable improvement for students or developers remains an empirical question.

5.5 Threats to Validity

5.5.1 Internal Validity

The same person developed and evaluated the prototype. This creates a confirmation-bias risk because the evaluator already knew the expected output and the intended interaction. No independent observer checked the scenario results.

5.5.2 Construct Validity

The criteria measure functional properties such as source linking, decomposition, and conflict marking. They are reasonable prerequisites for an explanatory tool, but they are not direct measurements of readability, debugging effectiveness,

learning, or usability. Passing the scenarios must not be interpreted as proof of those broader qualities.

5.5.3 External Validity

The evaluation uses a small set of Stella programs and selected language constructs. More complex combinations of extensions, deeply nested patterns, polymorphic types, and larger files may produce different results. The findings also cannot be generalized from Stella to industrial programming languages without further evaluation.

5.5.4 Conclusion Validity

There was no participant sample and no statistical analysis. The evaluation provides categorical pass/fail observations only. It cannot estimate an effect size or establish a causal relationship between visualization and task performance.

5.5.5 Reliability and Reproducibility

The checks were performed manually, and there is no complete automated end-to-end suite for the Visual Studio Code interactions. The included examples and expected outputs improve reproducibility, but future work should add protocol-level tests and UI automation. A tagged artifact version should also be used for any later user study.

5.6 Chapter Summary

The scenario evaluation confirms that the implemented views provide the intended structures, navigation, and conflict markers for the tested programs. It answers RQ2 at the functional level. The strongest remaining limitation is the

absence of a controlled user study; therefore, claims about improved readability or debugging speed are intentionally not made.

Chapter 6

Conclusion

This work designed and implemented a Visual Studio Code extension that exposes internal Stella analysis data through five coordinated views: an AST, a scope and typing-context panel, a type derivation tree, a reverse type-error trace, and a type dependency graph.

For RQ1, the implementation shows that Stella semantic information can be integrated without placing all information in one interface. Hierarchical structures are rendered with tree views, while proof-like and graph-based structures are rendered with webviews. Custom LSP requests preserve the separation between semantic computation on the server and presentation in the client. Shared source ranges connect every view to the original program.

For RQ2, the six author-executed scenarios showed that the prototype satisfied locality, hierarchical decomposition, source traceability, bidirectional navigation, and explicit conflict indication for the tested programs. The dependency graph additionally made the meaning of type relations explicit through labeled directed edges and conflict markers.

The evaluation does not confirm that the extension improves code readability, debugging speed, or learning outcomes for a representative user population. Such a conclusion would require a controlled user study. A suitable next experiment would compare standard Stella editor feedback with the developed exten-

sion in a within-subject design. Participants would complete code-understanding and type-error-localization tasks in both conditions. Completion time, correctness, navigation steps, and unsuccessful attempts would provide objective measures, while SUS could be used as a separate subjective usability measure [14], [15], [16].

Further technical work should expand derivation support for additional Stella extensions, improve error traces for more diagnostic categories, test dependency graphs on larger programs, and add automated protocol and UI tests. These steps would make the artifact more robust and provide a stronger basis for a later user study.

References

[1] B. C. Pierce, *Types and Programming Languages*. Cambridge, MA: MIT Press, 2002, ISBN: 978-0-262-16209-8.

[2] R. Harper, *Practical Foundations for Programming Languages*, 2nd ed. Cambridge University Press, 2016. DOI: 10.1017/CBO9781316576892

[3] Z. Gao, C. Bird, and E. T. Barr, "To type or not to type: Quantifying detectable bugs in JavaScript," in *Proceedings of the 39th International Conference on Software Engineering*, 2017, pp. 758–769. DOI: 10.1109/ICSE.2017.75

[4] S. Hanenberg, "An experiment about static and dynamic type systems: Doubts about the positive impact of static type systems on development time," in *Proceedings of the 2010 ACM International Conference on Object Oriented Programming Systems Languages and Applications*, 2010, pp. 22–35. DOI: 10.1145/1932682.1869462

[5] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006, ISBN: 978-0-321-48681-3.

[6] B. A. Price, R. M. Baecker, and I. S. Small, "A principled taxonomy of software visualization," *Journal of Visual Languages and Computing*, vol. 4, no. 3, pp. 211–266, 1993. DOI: 10.1006/jvlc.1993.1015

[7] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, “A meta-study of algorithm visualization effectiveness,” *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002. DOI: 10.1006/jvlc.2002.0237

[8] T. L. Naps et al., “Exploring the role of visualization and engagement in computer science education,” in *Proceedings of the Working Group Reports from ITiCSE on Innovation and Technology in Computer Science Education*, 2002, pp. 131–152. DOI: 10.1145/782941.782998

[9] J. Sorva, V. Karavirta, and L. Malmi, “A review of generic program visualization systems for introductory programming education,” *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–64, 2013. DOI: 10.1145/2490822

[10] G. Kogan, H. Chassidim, and I. Rabaev, “The efficacy of animation and visualization in teaching data structures: A case study,” *Educational Technology Research and Development*, vol. 72, no. 4, pp. 2349–2372, 2024. DOI: 10.1007/s11423-024-10382-w [Online]. Available: https://doi.org/10.1007/s11423-024-10382-w

[11] A. Moreno, N. Myller, E. Sutinen, and M. Ben-Ari, “Visualizing programs with Jeliot 3,” in *Proceedings of the Working Conference on Advanced Visual Interfaces*, 2004, pp. 373–376. DOI: 10.1145/989863.989928

[12] R. Aalvik, “visAST: Generic AST visualiser for software language education,” Accessed: June 19, 2026, M.S. thesis, University of Bergen, 2019. [Online]. Available: https://hdl.handle.net/1956/19781

[13] A. Abounegm, N. Kudasov, and A. Stepanov, “Teaching type systems implementation with STELLA, an extensible statically typed programming language,” *Electronic Proceedings in Theoretical Computer Science*, 2024,

Accessed: June 19, 2026. arXiv: 2407.08089 [cs.PL]. [Online]. Available: https://arxiv.org/abs/2407.08089

[14] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, I. L. McClelland, and B. Weerdmeester, Eds., London: Taylor & Francis, 1996, pp. 189–194.

[15] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009. [Online]. Available: https://uxpajournal.org/determining-what-individual-sus-scores-mean-adding-an-adjective-rating-scale/

[16] International Organization for Standardization, *ISO 9241-11:2018 ergonomics of human-system interaction – part 11: Usability: Definitions and concepts*, https://www.iso.org/standard/63500.html, 2018.

[17] J. Sorva, "Notional machines and introductory programming education," *ACM Transactions on Computing Education*, vol. 13, no. 2, pp. 1–31, 2013. DOI: 10.1145/2483710.2483713

[18] M.-A. D. Storey, "Theories, tools and research methods in program comprehension: Past, present and future," *Software Quality Journal*, vol. 14, no. 3, pp. 187–208, 2006. DOI: 10.1007/s11219-006-9216-4

[19] N. Kudasov, *Stella language documentation*, https://fizruk.github.io/stella/site/core/introduction/, Accessed: June 19, 2026, 2024.

[20] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Springer, 1999. DOI: 10.1007/978-3-662-03811-6

[21] Microsoft, *Language server protocol specification 3.17*, https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/, Accessed: June 19, 2026, 2024.

[22] JSON-RPC Working Group, *JSON-RPC 2.0 specification*, https://www.jsonrpc.org/specification, Accessed: June 19, 2026, 2013.

[23] Microsoft, *Visual studio code extension api*, https://code.visualstudio.com/api, Accessed: June 19, 2026, 2026.

[24] Microsoft, *Visual studio code extension api: Tree view*, https://code.visualstudio.com/api/extension-guides/tree-view, Accessed: June 19, 2026, 2026.

[25] Microsoft, *Visual studio code extension api: Webview*, https://code.visualstudio.com/api/extension-guides/webview, Accessed: June 19, 2026, 2026.

[26] T. R. G. Green and M. Petre, "Usability analysis of visual programming environments: A cognitive dimensions framework," *Journal of Visual Languages and Computing*, vol. 7, no. 2, pp. 131–174, 1996. DOI: 10.1006/jvlc.1996.0009

[27] C. Ware, *Information Visualization: Perception for Design*, 4th ed. Morgan Kaufmann, 2021, ISBN: 978-0-12-812875-6.

[28] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems*, vol. 9, no. 3, pp. 319–349, 1987. DOI: 10.1145/24039.24041

[29] M. Weiser, "Program slicing," in *Proceedings of the 5th International Conference on Software Engineering*, 1981, pp. 439–449.

[30] S. Horwitz, T. Reps, and D. Binkley, “Interprocedural slicing using dependence graphs,” *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 1, pp. 26–60, 1990. DOI: 10.1145/77606.77608

[31] C. Wohlin, P. Runeson, M. Hoest, M. C. Ohlsson, B. Regnell, and A. Wesslen, *Experimentation in Software Engineering*. Springer, 2012. DOI: 10.1007/978-3-642-29044-2

[32] P. Runeson and M. Hoest, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009. DOI: 10.1007/s10664-008-9102-8

[33] Microsoft, *The TypeScript handbook*, https://www.typescriptlang.org/docs/handbook/intro.html, Accessed: June 19, 2026, 2026.

[34] TypeFox, *Langium documentation*, https://langium.org/docs/, Accessed: June 19, 2026, 2026.

[35] TypeFox, *Typir: Type systems for language engineering*, https://github.com/typefox/typir, Accessed: June 19, 2026, 2026.

Appendix A

Artifacts and Reproduction

A.1 Artifact Location

The development repository is available at

`https://github.com/friji350/stella-lsp-extensions`.

The submitted thesis archive contains the exact source revision used for the screenshots and manual scenarios. Because the development repository may change, evaluation should use the submitted archive or a release tag associated with the thesis.

A.2 Build and Launch

The following commands are executed from the repository root:

1. Install dependencies with `npm install`.
2. Build the extension with `npm run build`.
3. Open the repository in Visual Studio Code.
4. Start an Extension Development Host and open one of the validation `.stella` files.

The commands above reproduce the implementation used for the manual scenarios. The Visual Studio Code commands exposed by the artifact are `Stella: Show Type Derivation`, `Stella: Show Type Error Trace`, `Stella: Show Type Dependency Graph`, and `Stella: Show AST (JSON)`.

A.3 Implementation Map

The main implementation artifacts are listed below.

- **AST visualization:** client: `src/extension/syntax-tree.ts`; server: `src/language/lsp/syntax-request.ts`.
- **Scope and context:** client: `src/extension/scope-view.ts`; server: `src/language/lsp/scope-request.ts`.
- **Type derivation:** client: `src/extension/type-derivation-panel.ts`; server: `src/language/lsp/type-derivation-request.ts`.
- **Type-error trace:** client: `src/extension/type-error-trace-panel.ts`; server: `src/language/lsp/type-error-trace-request.ts`.
- **Type dependency graph:** client: `src/extension/type-dependency-graph-panel.ts`; server: `src/language/lsp/type-dependency-graph-request.ts`.
- **Shared response structures:** `src/shared/lsp` and `src/shared/analysis`.

A.4 Validation Inputs

The validation files are included with the artifact:

- dependency-graph programs: `examples/dependency-graph`;
- expected dependency nodes and edges: `examples/type-dependency-graph-samples.md`;
- error-trace inputs and expected rules: `examples/error-trace-samples.md`.

To repeat a manual scenario:

1. Open a listed `.stella` file in the Extension Development Host.
2. Place the cursor on the expression named in the scenario description.
3. Run the corresponding `Stella: Show ...` command.
4. Compare nodes, type labels, edge labels, and conflict markers with the documented expectation.
5. Click visual elements and verify that the expected source ranges are revealed.

The screenshots used in Chapter 4 are stored in `Anton_Fadeenkov/figs`.